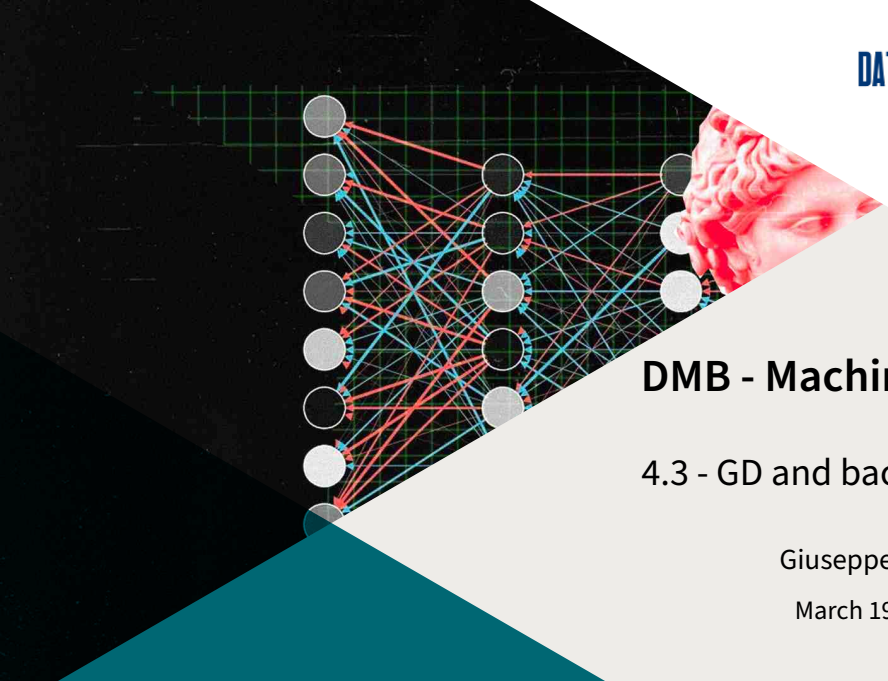


DMB - Machine Learning

4.3 - GD and backpropagation

Giuseppe Longo

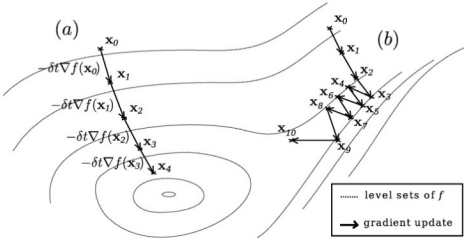
March 19, 2026

Overview

1. Gradient Descent
2. Stochastic Gradient Descent
3. The backpropagation algorithm
4. The four fundamental equations of BP
5. Combining GD and BP

Gradient Descent

Before starting I suggest you to watch **this movie**:



Different paths for the minimum.

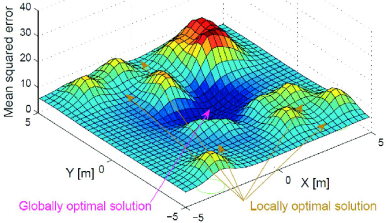
Our goal is to find the weigths' configuration which minimizes the cost function.

Two main things need to be kept in mind:

- The speed at which convergence is reached depends upon many factors such as: the point where you start exploring, the shape of the surface, the step you choose to explore the surface, ecc.

For instance in the image, you see two different paths: (a) is well behaved and converges quickly, (b) gets trapped in a narrow valley and oscillates thus slowing down the convergence.

The Cost Hypersurfaces



- The cost hypersurface of a problem with N (number of weights) cannot be visualized.

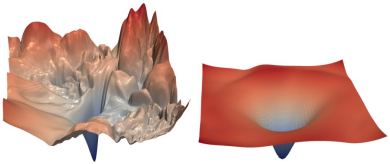
  We can use as an example only N=2 cases (e.g. the non convex mean squared error (cost function) in Fig.)

  - Only in some cases you have convex surfaces with only one minimum. In almost all problemìs cost surfaces are non-convex (like in the figure) and have many local minima and many local maxima.
  - **Global Minimum**. The absolute lowest point on the entire surface, representing the best possible parameter settings.
  - **Local Minima**. "False bottoms" where an optimization algorithm like Gradient Descent might get stuck, thinking it has found the best solution when a better one exists elsewhere (in some cases this might be irrelevant).

- in real DL problems with billion of parameters researchers use dimensionality reduction techniques to project these hypersurfaces in 2D/3D. Some methods have been developed.

  - **filter normalization**.
  - **t-SNE or UMAP**
  - **Trajectory Plotting**
  - **Hessian analysis**.

The Filter Normalization

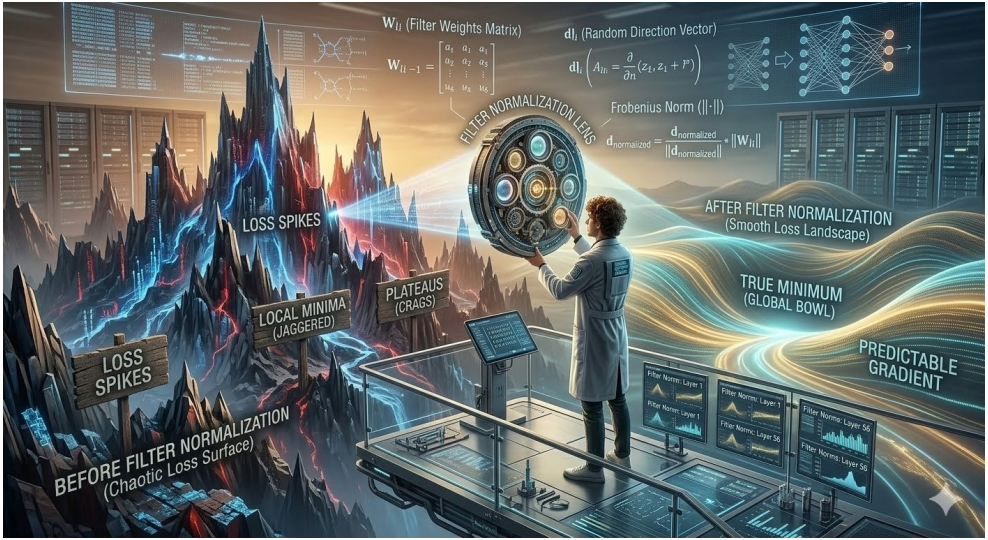


(a) without skip connections (b) with skip connections

- If we pick two directions (vectors) in the parameter space you can see how the loss changes along them. There is however a problem: **Scale invariance.**
- In most neural networks (especially with Batch Normalization), you can multiply the weights of a layer by 10 and divide the next layer's weights by 10 without changing the network's output. However, the loss surface will appear 10 times "steeper" or "shaper" simply because the numbers are larger, not because the model is actually harder to train.

Filter normalization solves this problem:

- Start at the Center: Imagine your trained model's weights as a single point in a million-dimensional space. We call this point θ^*.
- Pick Two Directions at random (vectors=directions): δ and η. These will represent our X and Y axes for the 3D plot.
- Normalization Step: Instead of just moving in those random directions, we scale the vectors to match the "size" (the norm) of the weights in each specific filter of the neural network. If a specific convolutional filter has very small weights, we move it only a tiny bit. If a filter has large weights, we move it more.
- Calculate the Grid: We create a grid of points around our center. At each point (x, y) on our grid, we calculate the loss of the model using a new set of weights: $\theta = \theta^* + x\delta + y\eta$.
- Plot the Results: We take all those loss values and plot them as the height (Z-axis).


$\mathbf{W}_{li}$ (Filter Weights Matrix)
$\mathbf{W}_{li-1} = \begin{bmatrix} a_1 & a_1 & a_1 \\ a_2 & a_2 & a_3 \\ \vdots & \vdots & \vdots \\ u_4 & u_2 & u_6 \end{bmatrix}$
$\mathbf{dl}_i$ (Random Direction Vector)
$\mathbf{dl}_i \left(A_{li} = \frac{\partial}{\partial n}(z_1, z_1 + P^*) \right)$
Frobenius Norm (||·||)
$\mathbf{d}_{normalized} = \frac{\mathbf{d}_{normalized}}{||\mathbf{d}_{normalized}||} * ||\mathbf{W}_{li}||$
FILTER NORMALIZATION LENS
LOSS SPIKES
LOCAL MINIMA (JAGGERED)
PLATEAUS (CRAGS)
LOSS SPIKES
BEFORE FILTER NORMALIZATION (Chaotic Loss Surface)
AFTER FILTER NORMALIZATION (Smooth Loss Landscape)
TRUE MINIMUM (GLOBAL BOWL)
PREDICTABLE GRADIENT
Filter Norms: Layer 1
Filter Norms: Layer 1
Filter Norms: Layer 56
Filter Norms: Layer 56
Filter Norms: Layer 56
Filter Norms: Layer 56

Gradient Descent or **GD** is a numerical technique used to find the minimum of a function of many variables, in this case: the function will be the **Cost** function C.

By exploiting the derivative of the Cost function C expressed as a function of the weigths vector, $C(\mathbf{w})$, the **GD** allows corrective updates to the learned estimates that move the model towards an optimal combination of weights.

In a machine learning algorithm, the cost is calculated over the entire training dataset for each iteration of the gradient descent (and backpropagation, see later) algorithm.

The variation of the cost function C as a result of the variation in the weigths may be written as:

$$\Delta C \sim \sum_{i=1}^{n} \frac{\partial C}{\partial w_i} \Delta w_i$$

where n is the number of weights.

link between ΔC and $\Delta \mathbf{W}$

We shall denote the gradient of C as ∇C.

The variation ΔC of the cost function c can then be written as:

$$\Delta C \sim \nabla \mathbf{C} \cdot \Delta \mathbf{w}$$

where

$$\nabla \mathbf{C} = (\frac{\partial C}{\partial w_1}, ... \frac{\partial C}{\partial w_n})^T$$

is the gradient column vector, and $\Delta \mathbf{w} = (\Delta w_1, ... \Delta w_n)^T$ is the column vector containing the variations of the weights.

simple but rigorous trick

we introduce a small positive parameter η and pose that $\Delta \mathbf{w} = -\eta \nabla \mathbf{C}$.

In this way we make sure that $\Delta C < 0$ since

$$\Delta \mathbf{C} \sim -\eta \nabla \mathbf{C} \cdot \nabla \mathbf{C} = -\eta ||\nabla C||^2$$

The equation which we were looking for hance becomes:

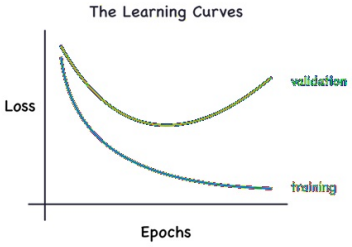
$$\mathbf{w}' = \mathbf{w} - \eta \nabla \mathbf{C}$$

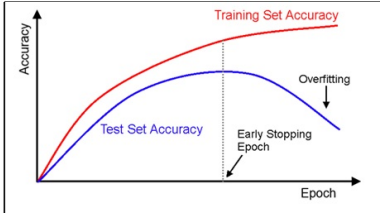
.

where η is called **Learning Rate**.

The above is our **Weights updating rule**.

General considerations





- A larger step size (i.e. larger learning rate) allows faster convergence but increases the chance to get trapped in a local minimum and increases the probability that the path oscillates;
- smaller learning rates ensure more proper descent but slow down convergence

Questions:

- How do we know that we have achieved some sort of convergence and that results cannot improve anymore?
- How do we know when to stop the process?

- The simplest way is to monitor the value of the cost and use any of the following

  options (or all of them):
  - after a predefined number of steps,....
  - ... when the cost function has not decreased for some time.
  - ... when the network starts overfitting
  - .... **IF YOU HAVE ACHIEVED THE DESIRED THRESHOLD YOU CAN STOP AND HAND IN THE PROJECT.**

Important

While the computation time strongly depends on the number of parameters, gradient descent does not suffer much from the curse of dimensionality and actually benefits from a higher number of dimensions.

GD in fact tends to get stuck in narrow valleys or plateaus where the derivative is zero. In spaces of high dimension, these issues are much less common because every additional dimension increases the possibility of finding a way out.

Important, 2

The gradient descent algorithm depends on the possibility to compute the partial derivatives at each step, it can be applied only when an explicit formula is available for the objective function, which is not always the case.

An alternative is to use 2-nd order information from the Hessian, e.g. using Newtons method, or Hessian-Free optimization.

Obviously, when the number of weights explodes, computation may become an issue and other optimization strategies need to be explored. But for the moment we shall restrict ourselves to GD and **Stochastic Gradiend Descent** or **SGD**.

You can find some very interesting considerations on GD **here**.

Stochastic Gradient Descent

To compute ∇C we need to compute $\nabla C_{\mathbf{x}}$ for each of the n input records and then take the average: $C = \frac{1}{n} \sum_x C_x$.

This can be very expensive in terms of computing.

To speed up computations it is possible to estimate the gradient by **using small samples** S **of** m **randomly extracted training examples**. If m is large enough to properly sample the dataset:

$$\frac{\sum_x \nabla \mathbf{C_x}}{n} \sim \frac{\sum_{j=1}^{m} \nabla \mathbf{C_{X_s}}}{m} \sim \nabla \mathbf{C}$$

and:

$$\nabla \mathbf{C} \sim \frac{1}{m} \sum_{j=1}^{m} \nabla \mathbf{C_{X_j}}$$

Algorithm

Then the algorithm for updating the weights can be summarised as it follows:

- select m samples to form a mini-batch, then compute the above quantity
- update the weights using the following rules:
  $w'_k = w_k - \frac{\eta}{m} \sum_j \frac{\partial C_{X_j}}{\partial w_k}$ $b'_l = b_l - \frac{\eta}{m} \sum_j \frac{\partial C_{X_j}}{\partial b_l}$
- pick up another randomly selected mini-batch and repeat until you have exhausted the training set. For extension, each iteration of the weight updating algorithm is often called a **batch**.
- The **WHOLE** sequence over all batches defines **an epoch**
- Reiterate for a new epoch until you reach convergence.

WARNINGS

- We scaled the overall cost function by a factor 1/n. People sometimes omit the 1/n, summing over the costs of individual training examples instead of averaging. This is particularly useful when the total number of training examples is not known in advance. **This can occur if more training data is being generated in real time, for instance**.

- In a similar way, the mini-batch update rules sometimes omit the 1/m term out the front of the sums. Conceptually this makes little difference, since it's equivalent to rescaling the learning rate η. But when doing detailed comparisons of different work it's worth watching out for it.

The backpropagation algorithm

Backpropagation Method

Backpropagation is the golden egg chicken of Machine Learning

It is a method to update the weights towards the direction (gradient) that minimizes a predefined error metric (i.e. the Loss Function) given a batch of labeled observations.

This algorithm has been repeatedly rediscovered and is a special case of a more general technique called *automatic differentiation* in *reverse accumulation mode*.

Its relevance to ML, however, dates back to a 1986 **paper by David Rumelhart, Geoffrey Hinton, and Ronald Williams**. They described several neural networks where backpropagation works far faster than earlier methods thus making it possible to use neural nets to solve problems which had previously been insoluble.

As mentioned before, neural networks can be represented as computational graphs, hence from a mathematical point of view they behave like composite functions.

To understand how the system works you must keep in mind that the workings of a network can be split in two parts:

- **forward phase**: inputs from a training sample are cascaded through the layers using the existing set of weights and produce an output.
- **backward phase**: learns the gradient of the cost function (with respect to the weights) using the chain rule of differentiation for composite functions.

Using the algebraic notation introduced previously, we wish to point out that the problem arises from the fact that in order to perform GD or SGD we need to evaluate the gradient $\frac{\partial C_i}{\partial W_{ij}^l}$ quickly and ideally in parallel, as this calculation can be performed many, many (billions of) times.

So we have:

$$a_j^l = \sigma\left(\sum_k w_{jk}^l a_k^{l-1} + b_j^l\right)$$

where the sum is over all neurons k in the $(l1)^{th}$ layer. In matrix notation, this becomes:

$$\mathbf{a}^l = \sigma(\mathbf{W}^{l^T}\mathbf{a}^{l-1} + \mathbf{b}^l)$$

Let us again introduce the **weighted input** to the neurons in the layer l:

$$\mathbf{z}^l \equiv \mathbf{W}^{l^T}\mathbf{a}^{l-1} + \mathbf{b}^l$$

needless to say:

$$\mathbf{a}^l = \sigma(\mathbf{z}^l)$$

$$z_j^l = \sum_k w_{jk}^l a_k^{l1} + b_j^l$$

that is, z_j^l is just the weighted input to the activation function for neuron j in layer l.

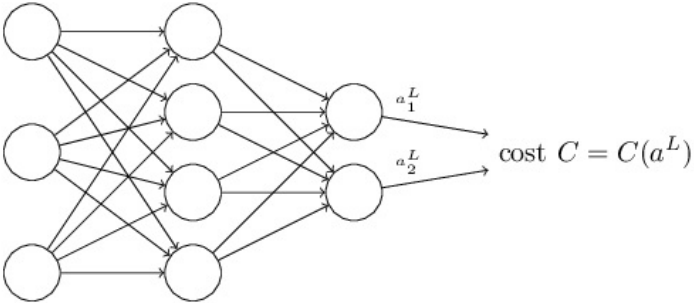
This notation helps in understanding some properties of the cost function C and of the backpropagation algorithm.

Backpropagation requires the existence of the partial derivatives $\partial C/\partial w$ and $\partial C/\partial b$ with respect to any weight w or bias b in the network. For backpropagation to work we need therefore to make two main assumptions about the form of the cost function C.

- **First assumption on** C : it can be written as an average $C = \frac{1}{n}\sum_x C_x$ over the cost functions C_x for individual training examples, x.
- **Second assumption on** C: the cost function needs to be written as a function of the outputs from the neural network.

The first assumption is justified because for BP we need to compute partial derivatives for a single training example and only afterwards recover $\partial C/\partial w$ and $\partial C/\partial b$ by averaging over training examples.

In fact, with this assumption in mind, we'll suppose the training example x has been fixed, and drop the x subscript, writing the cost C_x as C.



An example: the quadratic cost function

Let us take the case of the quadratic cost function:

$$C = \frac{1}{2n} \sum_{\mathbf{x}_1}^{\mathbf{x}_n} ||y(\mathbf{x})a^L(\mathbf{x})||^2$$

where: n is the total number of training examples $\mathbf{x}_n$; the sum is over individual training examples, $\mathbf{x}$; $y = y(\mathbf{x})$ is the corresponding desired output; L denotes the number of layers in the network; and $a^L = a^L(\mathbf{x})$ is the vector of activations output from the network when $\mathbf{x}$ is input.

Hence, the quadratic cost function, **fulfills the first assumption** since for a single training example is

$$C_x = \frac{1}{2}||ya^L||^2$$

It also **fulfills the second assumption** since:

$$C = \frac{1}{2}||y - a^L||^2 = \frac{1}{2}\sum_j (y_j - a_j^L)^2$$

The four fundamental equations of BP

Backpropagation (BP) is about understanding how changing the weights and biases in a network changes the cost function.

Ultimately, as we said, this means computing the partial derivatives $\partial C / \partial w^l_{jk}$ and $\partial C / \partial b^l_j$.

To compute those, we first introduce an intermediate quantity, δ^l_j, which we call the **error in the j-th neuron in the l-th layer**.

$$\delta^l_j = \frac{\partial C}{\partial z^l_j}$$

Backpropagation gives us a procedure to compute the error δ^l_j, and then to relate δ^l_j to the derivatives.

BP1 - first equation (error in the output layer L)

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L)$$

The quantity $\frac{\partial C}{\partial a_j^L}$ measures how fast the cost is changing as a function of the j-th output activation. If, for example, C does not depend much on a particular output neuron j, then δ_j^L will be small.

$\sigma'(z_j^L)$ instead measures how fast the activation function σ is changing at z_j^L.

We compute z_j^L while computing the behaviour of the network, and it is only a small additional overhead to compute $\sigma'(z_j^L)$.

The exact form of $\frac{\partial C}{\partial a_j^L}$ will, of course, depend on the form of the cost function. However, provided the cost function is known there should be little trouble computing it.

Let us now rewrite the main equation in matricial form:

$$\delta^L = \nabla_{\mathbf{a^L}} C \odot \sigma'(\mathbf{z}^L)$$

Where, $\nabla_{\mathbf{a^L}} C$ is the **gradient vector** whose components are the partial derivatives $\partial C / \partial a_j^L$.

You can think of $\nabla_{\mathbf{a^L}} C$ as expressing the rate of change of C with respect to the output activations.

For instance, in the case of the quadratic cost, we have $\nabla_{\mathbf{a^L}} C = (a^L y)$, and so the fully matrix-based form becomes

$$\delta^L = (a^L - y) \odot \sigma'(z^L)$$

Proof of BP1

Recall that by definition:

$$\delta_j^L = \frac{\partial C}{\partial z_j^L}$$

By applying the chain rule, the above partial derivative can be re-expressed in terms of partial derivatives with respect to the output activations,

$$\delta_j^L = \sum_k \frac{\partial C}{\partial a_k^L} \frac{\partial a_k^L}{\partial z_j^L}$$

where the sum is over all neurons k in the output layer.

Since the output activation a_k^L of the k-th neuron depends only on the weighted input z_j^L for the j-th neuron when $k = j$, $\partial a_k^L / \partial z_j^L$ vanishes when $k \neq j$.

As a result the previous equation becomes:

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \frac{\partial a_j^L}{\partial z_j^L}$$

Recalling that $a_j^L = \sigma(z_j^L)$ the second term on the right can be written as $\sigma'(z_j^L)$ and then, QED, we get the BP1 equation in component form:

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L)$$

BP2 - second equation

Gives the error δ^l in terms of the error in the next layer

in matricial form it is expressed as:

$$\delta^l = (\mathbf{W}^{(l+1)\top}\delta^{l+1}) \odot \sigma'(\mathbf{z}^l)$$

where $\mathbf{W}^{(l+1)\top}$ is the transpose of the weight matrix $\mathbf{W}^{(l+1)}$ for the $(l+1)^{th}$ layer.

Suppose we know the error $^{(l+1)}$ at the $(l+1)^{th}$ layer. When we apply the transpose weight matrix, $\mathbf{W}^{(l+1)\top}$, this has the effect of moving the error backward through the network, giving us some sort of measure of the error at the output of the lth layer.

We then take the Hadamard product $\odot\sigma'(z^l)$. This moves the error backward through the activation function in layer l, giving us the error δ^l in the weighted input to layer l.

NOTICE The first two equations allow us to compute the error δ^l for any layer in the network. By using the first one (BP1) we first compute δ^L, then apply the second equation (BP2) to compute δ^{L-1}, and then BP2 again to compute δ^{L-2}, and so on moving back through the network.

BP3 - third equation

It gives the rate of change of the cost with respect to any bias in the network:

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l$$

Which means that the error δ_j^l is exactly equal to the rate of change $\frac{\partial C}{\partial b_j^l}$.

In matricial form:

$$\frac{\partial C}{\partial b} = \delta$$

where it is meant that δ is evaluated at the same neuron as the bias b.

BP4 - fourth equation

This equation for the rate of change of the cost with respect to any weight in the network

$$\frac{\partial C}{\partial w_{jk}^{l}} = a_k^{l-1} \delta_j^l$$

This allows to compute the partial derivatives $\frac{\partial C}{\partial w_{jk}^{l}}$ in terms of the quantities δ^l and a^{l1}, which we already know how to compute.

In a simpler notation, we could write:

$$\frac{\partial C}{\partial w} = a_{in} \delta_{out}$$

where a_{in} is the activation of the neuron input to the weight w, and δ_{out} is the error of the neuron output from the weight w.

The four equations of BP

$$\delta^L = \nabla_{a^L} C \odot \sigma'(\mathbf{z}^L) \tag{1}$$

$$\delta^l = ((\mathbf{W}^{l+1})^\top \delta^{l+1}) \odot \sigma'(\mathbf{z}^l) \tag{2}$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \tag{3}$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \tag{4}$$

Initial implications

1 - BP4 implies that if the activation a_{in} is small,the gradient term $\partial C/\partial w$ will also tend to be small.

This means that the weight learns slowly, that is, in this case it does not change much during the gradient descend. In other words, one consequence of equation BP4 is that **weights output from low-activation neurons learn slowly**.

2 - Consider now the output layer. Start from $\sigma'(z_j^L)$ in BP1. Recall that logistic functions such as the sigmoid, become flat when $\sigma(z_j^L)$ is ~ 0 or ~ 1. Then: $\sigma'(z_j^L) \sim 0$.

Hence **a weight in the final layer will learn slowly if the output neuron is either in low activation (~ 0) or in high activation (~ 1)**. In other words, the output of that neuron **saturates** and, as a result, the weight either stops learning or learns very slowly. The same is true for the biases of any output neuron.

3 - Consider the $\sigma'(z^l)$ term in BP2. This implies that δ_j^l is likely to get small if the neuron is near saturation. Consequently, that **any weights input to a saturated neuron will learn slowly**.

Warning: This reasoning does not hold if $(W^{l+1})^T \delta^{l+1}$ has large enough entries to compensate for the smallness of the derivative $\sigma(z_j^l)$.

The BP algorithm

Step 1: input $\mathbf{x}$; set the corresponding activation a^1 for the input layer.

Step 2: Feedforward; for each $l = 2, 3, , L$ compute $\mathbf{z}^l = \mathbf{W}^l a^{l1} + \mathbf{b}^l$ and $\mathbf{a}^l = \sigma(\mathbf{z}^l)$.

Step 3: Output error δ^L; compute the vector $\delta^L = \nabla_{a^L} C \odot \sigma'(\mathbf{z}^L)$.

Step 4: Backpropagate the error; for each layer $l = L1, L2, , 2$ compute $\delta^l = ((\mathbf{W}^{l+1})^\top \delta^{l+1}) \odot \sigma(\mathbf{z}^l)$.

Step 5: Output; the gradient of the cost function is given by $\frac{\partial c}{\partial w^l_{jk}} = a^{l1}_k \delta^l_j$ and $\frac{\partial c}{\partial b^l_j} = \delta^l_j$.

Now it is clear why it is called backpropagation. Error vectors δ^l are computed backwards, starting from the final layer. The backward movement results from the fact that the cost is a function of outputs from the network.

Combining GD and BP

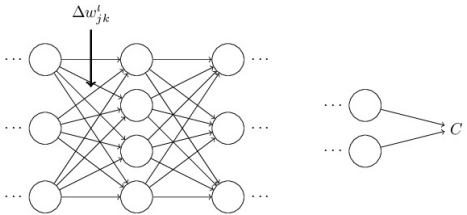
In practice, backpropagation is combined with a learning algorithm such as gradient descent, in which the gradient is computed over many training examples.

For each training example $\mathbf{x}$; set the corresponding input activation $\mathbf{a}^1(\mathbf{x})$, and perform the following steps:

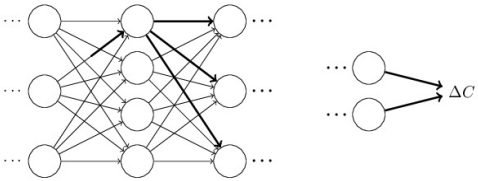
- **Step 1 - Feedforward**: for each layer $l = 2, 3, , L$ compute $\mathbf{z}^l(\mathbf{x}) = \mathbf{W}^l\mathbf{a}^{l1}(\mathbf{x}) + \mathbf{b}^l$ and $\mathbf{a}^l(\mathbf{x}) = \sigma(\mathbf{z}^l(\mathbf{x}))$.
- **Step 2 - Compute the output error** $\delta^{x,L}$: compute the vector $\delta^{x,L} = \nabla_{a^L} C_x \odot \sigma'(z^{x,L})$.
- **Step3 - Backpropagate the error**: for each $l = L1, L2, , 2$ compute $\delta^{x,l} = ((w^{l+1})^T \delta^{x,l+1}) \odot \sigma'(z^{x,l})$.
- **Step 4 - Gradient descent**: for each $l = L, L1, , 2$ update the weights according to the rule $w^l \rightarrow w^l \frac{\eta}{m} \sum_x {}^{x,l}(a^{x,l1})^\top$, and the biases according to the rule $b^l \rightarrow b^l \frac{\eta}{m} \sum_x \delta^{x,l}$.

In order to implement stochastic gradient descent are also needed: an outer loop generating mini-batches of training examples and an outer loop stepping through multiple epochs of training.

The logic behind BP



Suppose we've made a small change δw^l_{jk} to some weight in the network, w^l_{jk}, and follow the propagation. The change ΔC in the cost is related to the change δw^l_{jk} in the weight by the equation $\delta C \sim \frac{\partial C}{\partial w^l_{jk}} \delta w^l_{jk}$. This suggests that a possible approach to computing $\partial C / \partial w^l_{jk}$ is to carefully track how a small change in w^l_{jk} propagates to cause a small change in C. If we can do that, being careful to express everything along the way in terms of easily computable quantities, then we should be able to compute $\partial C / \partial w^l_{jk}$.

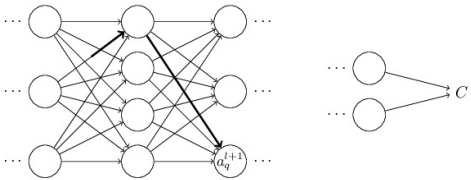


The change δw^l_{jk} causes a small change δa^l_j in the activation of the jth neuron in the lth layer:

$$\delta a^l_j \sim \frac{\partial a^l_j}{\partial w^l_{jk}} \delta w^l_{jk}$$

The change in activation δa^l_j will cause changes in all the activations in the next layer, i.e., the $(l+1)^{th}$ layer.

Let's focus on the way just a single one of those activations is affected, say a_q^{l+1}



This will cause the following change:

$$\delta a_q^{l+1} \sim \frac{\partial a_q^{l+1}}{\partial a_j^l} \delta a_j^l$$

Substituting in the previous equation we get:

$$\delta a_q^{l+1} \sim \frac{\partial a_q^{l+1}}{\partial a_j^l} \frac{\partial a_j^l}{\partial w_{jk}^l} \delta w_{jk}^l$$

Of course, the change δa_q^{l+1} will, in turn, cause changes in the activations in the next layer.

In fact, we can imagine a path all the way through the network from w^l_{jk} to C, with each change in activation causing a change in the next activation, and, finally, a change in the cost at the output.

If the path goes through activations $a^l_j, a^{l+1}_q, , a^{L1}_n, a^L_m$ then the resulting expression is

$$\delta C \sim \frac{\partial C}{\partial a^L_m} \frac{\partial a^L_m}{\partial a^{L-1}_n} \frac{\partial a^{L-1}_n}{\partial a^{L-2}_p} \cdots \frac{\partial a^{l+1}_q}{\partial a^l_j} \frac{\partial a^l_j}{\partial w^l_{jk}} \delta w^l_{jk}$$

which means that a $\partial a / \partial a$ term has been taken for each additional neuron we've passed through, including the $\partial C / \partial a^L_m$ term at the end.

This represents the change in C due to changes in the activations along a particular path through the network.

To compute the total change in C we should sum over all the possible paths between the weight and the final cost.

The other side of the coin

From the above it is apparent that the bigger the gradient, the bigger the adjustments and vice versa. Here is where the **vanishing gradient problem** lies.

When doing back propagation, each node in a layer calculates its gradient with respect to the effects of the gradients in the layer before it.

Let us suppose that for an early layer the average gradient $\Delta C/\delta w \sim 10^{-15}$, given a learning rate η of 10^{-4}, you basically change the layer parameters by the product of the referenced quantities, which is $\Delta w_i \sim 10^{-19}$.

As a results, you dont actually observe any change in the model while training your network.

In general, multiplication with absolute value less than 1 is nice because it provides some sense of training stability, although there is not a strict mathematic theorem about that. However, one can observe that for every layer that we go backwards in the network the gradient of the network gets smaller and smaller. **This can be a serious problem in multi-layers network**. Other problems will become more evident later on, and

they are:

- **The exploding gradient problem**
- **Difficulties in convergence**
- **Local and spurious minima**.

Exercises

Exercise n.1

- Prove BP2, BP3 and BP4

Exercise n.2

- Implement backpropagation with a single modified neuron. Suppose we modify a single neuron in a feedforward network so that the output from the neuron is given by $f(\sum_j w_j x_j + b)$, where f is some function other than the sigmoid. How should we modify the backpropagation algorithm in this case

Exercise n.3

- Backpropagation with linear neurons Suppose we replace the usual non-linear σ function with $\sigma(z) = z$ throughout the network. Rewrite the backpropagation algorithm for this case.